



<https://haoailab.com/cse291-s26/>

CSE/DSC 291: Deep Learning Systems Spring 2026

LLM, diffusion, and case studies

Optimizations and Parallelization

Basics

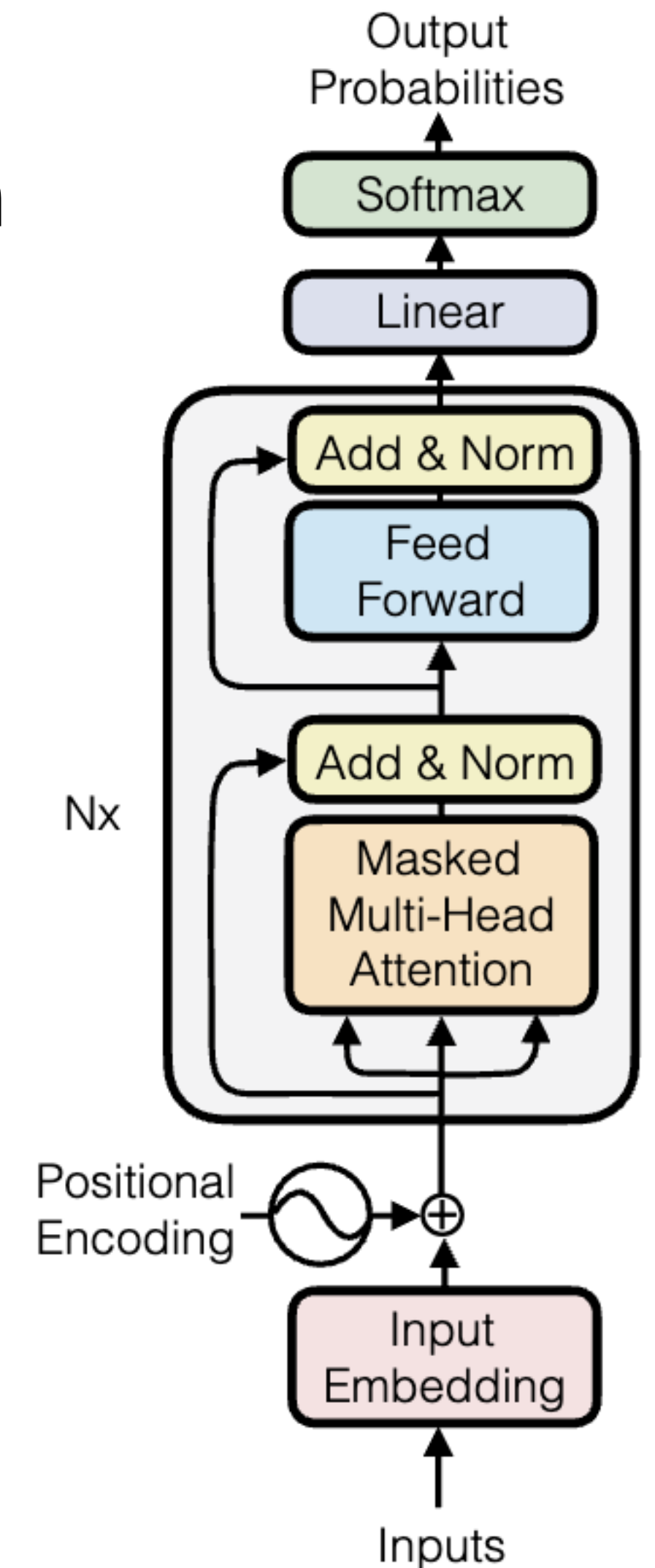
Logistics

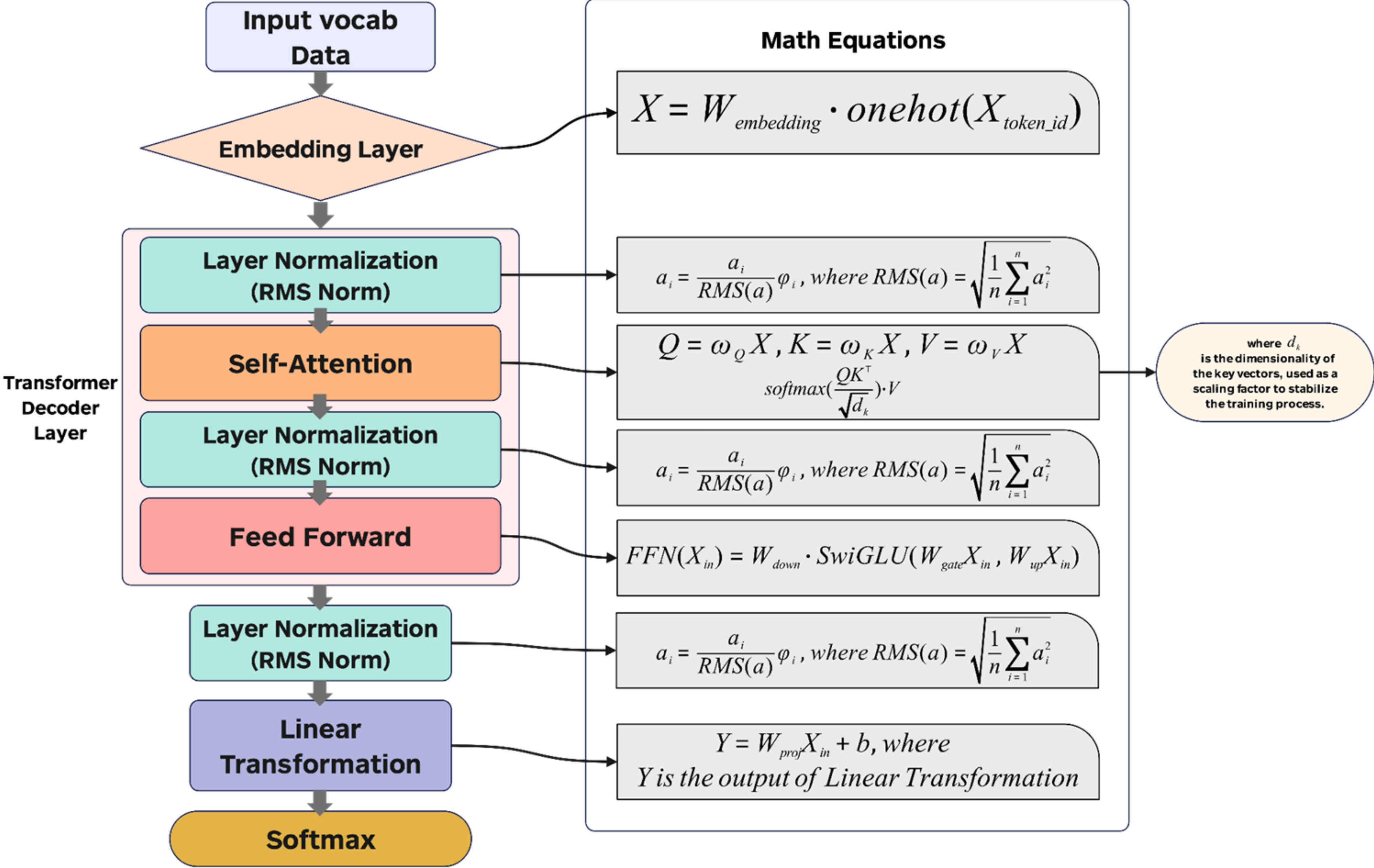
- PA3 was released, enjoy!
- Next Tuesday (Zoom): Flash attention and training optimization
- Next Thursday (Guest Lecture): Zihao Ye -- flashinfer creator

Connecting the Dots: Compute/Comm characteristic of LLMs

Key characteristics: compute, memory, communication

- calculate the number of parameters of an LLM?
- calculate the flops needed to train an LLM?
- calculate the memory needed to train an LLM?





Feed Forward SwiGLU

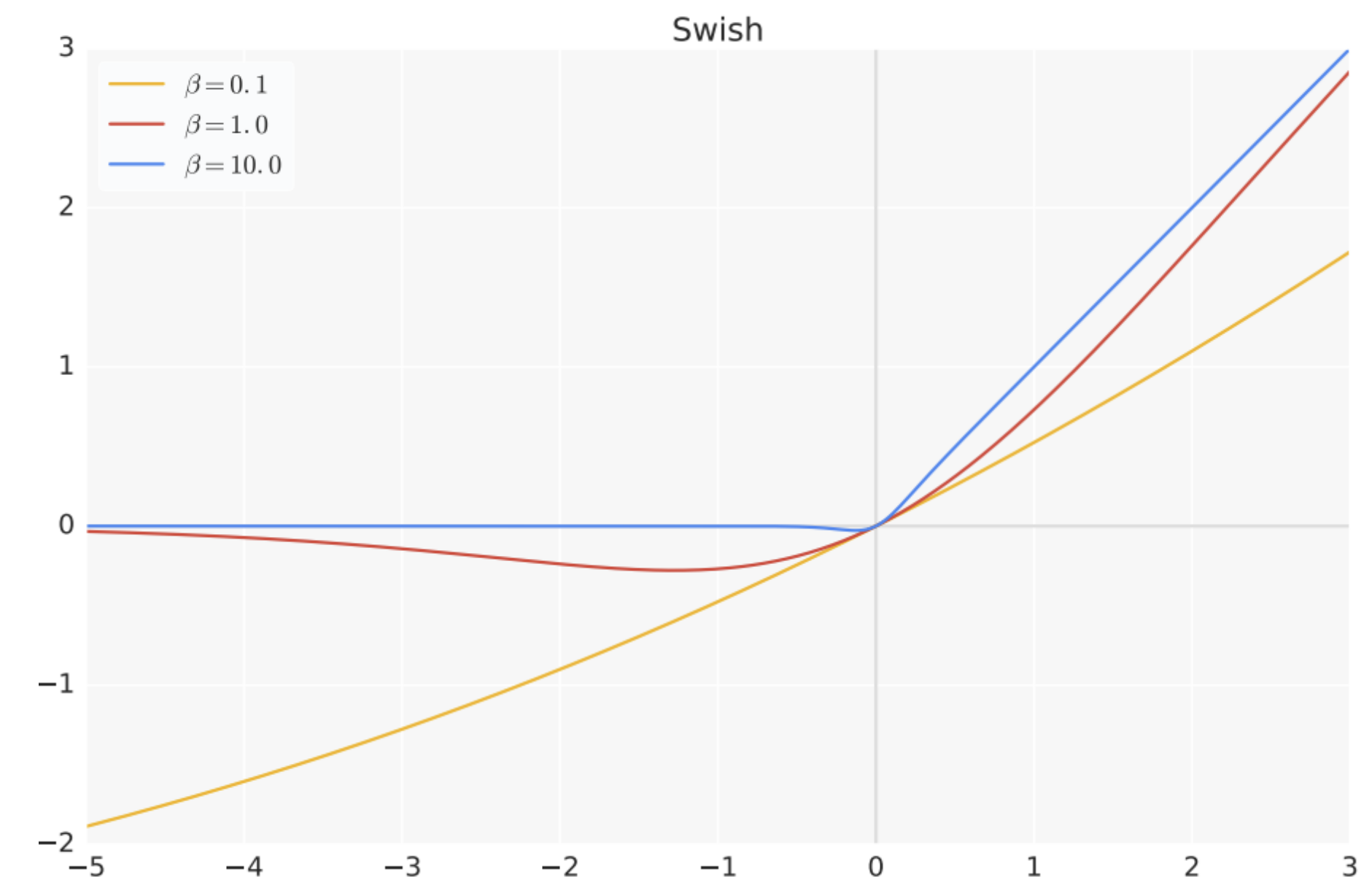
The general formula for SwiGLU is:

$$\text{SwiGLU}(x) = \text{Swish}(xW_1 + b_1) \odot (xW_2 + b_2)$$

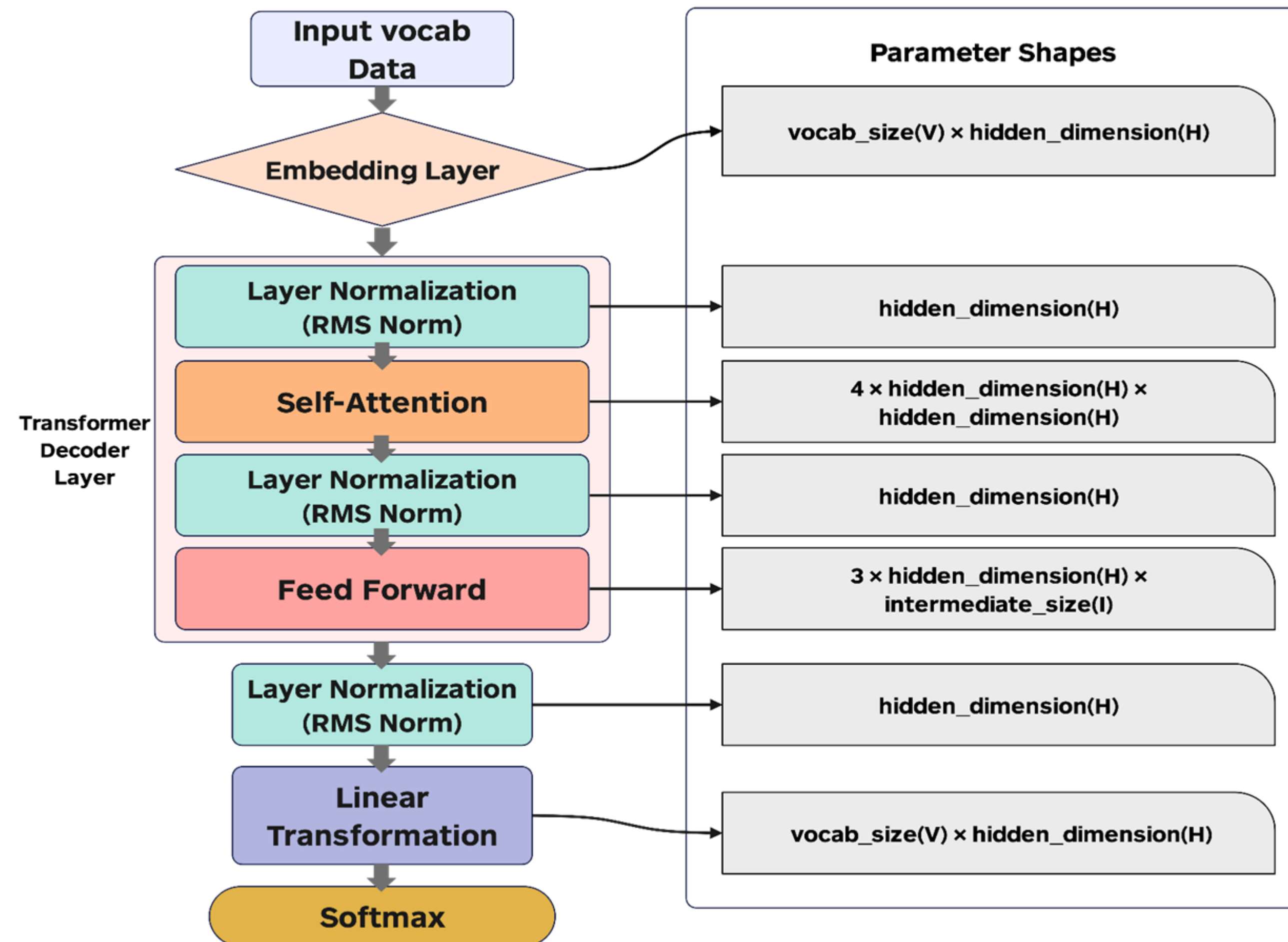
Swish is the activation function applied to one branch, defined as:

$$\text{Swish}(z) = z \cdot \sigma(z)$$

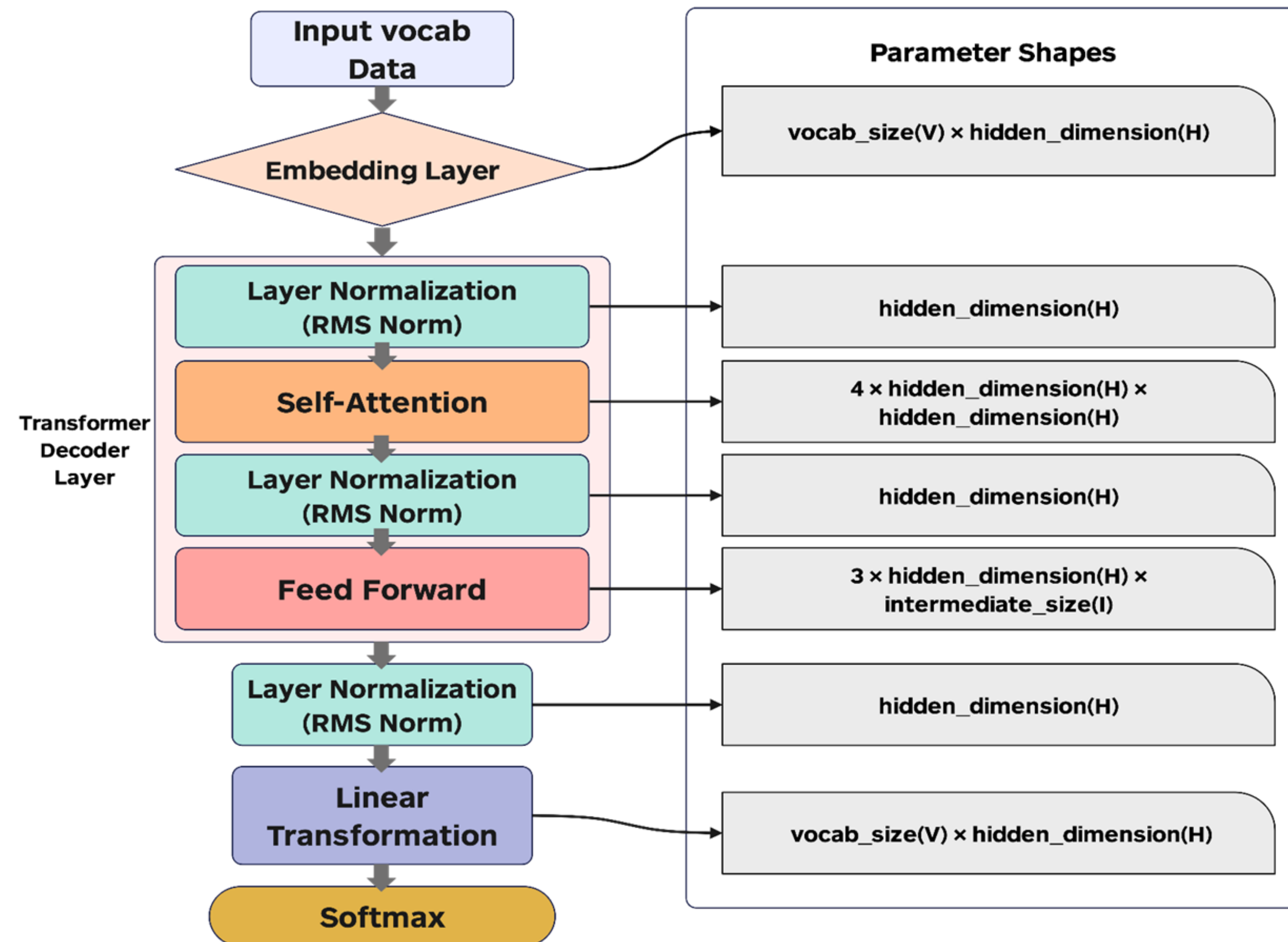
- SwiGLU helps the model capture more complex patterns by selectively gating information
- Swish is smoother than traditional activations ReLU



Summary



Scaling Up: Where is the Potential Bottleneck?

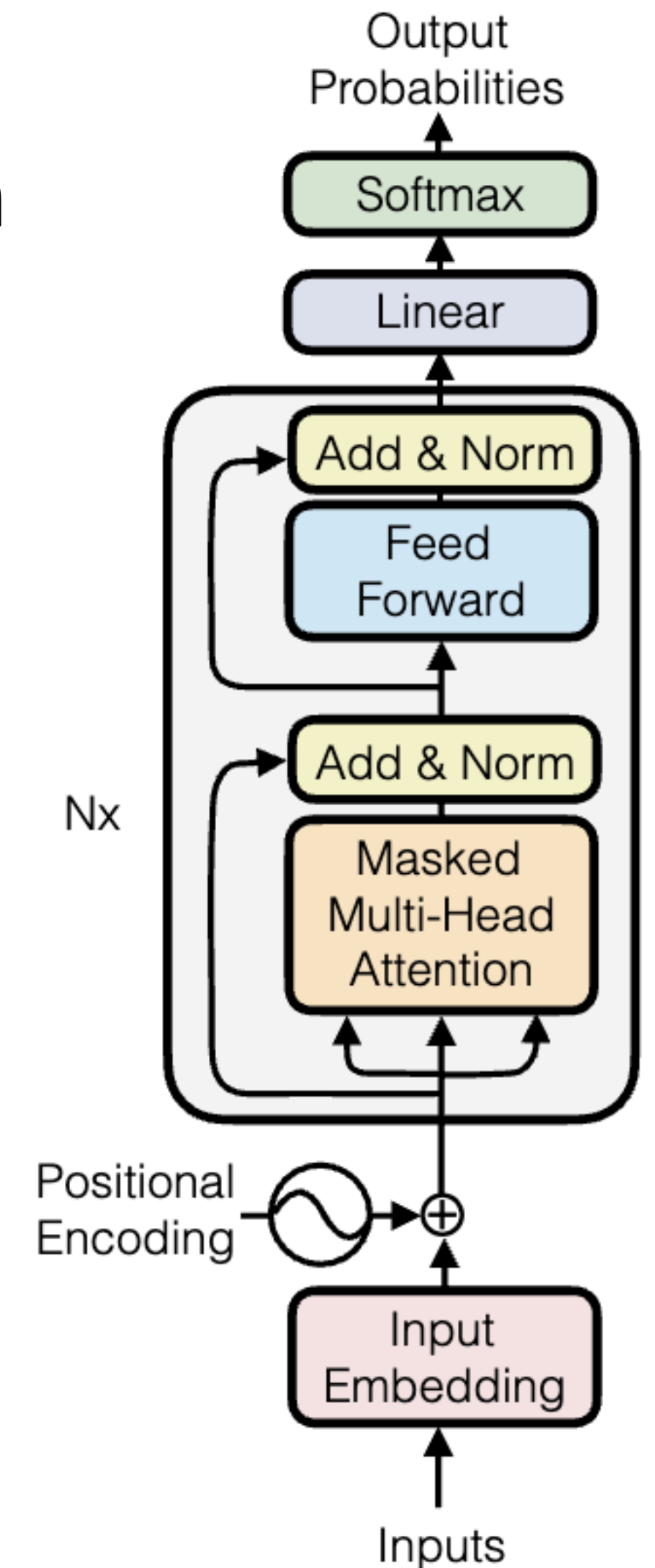


In PA3, you will implement this function 😊

Connecting the Dots: Compute/Comm characteristic of LLMs

Key characteristics: compute, memory, communication

- calculate the number of parameters of an LLM?
- calculate the flops needed to train an LLM?
- calculate the memory needed to train an LLM?

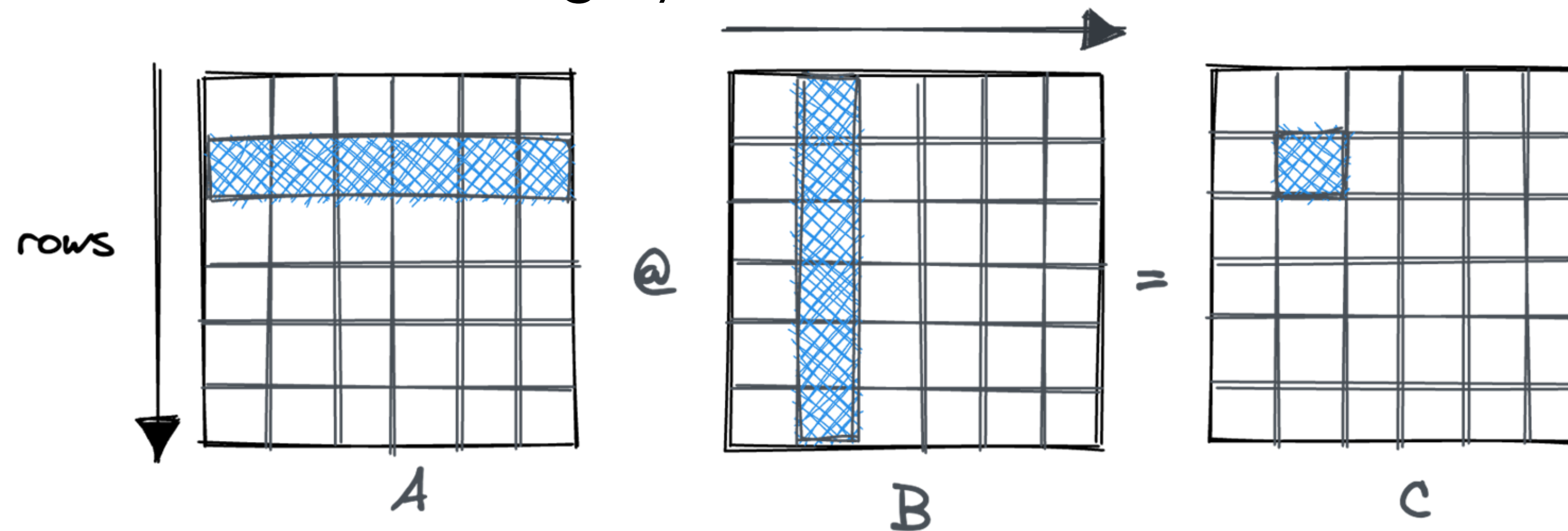


Estimate the Compute: FLOPs

The FLOPs for multiplying two matrices of dimensions $m \times n$ and $n \times h$ can be calculated as follows:

$$\text{FLOPs} = m \times h \times (2n - 1)$$

So the total number of FLOPs is roughly $\text{FLOPs} \approx 2m \times n \times h$



LLama 2 7B Flops Forward Calculation (Training)

Hyperparameters:

Batch size: b

Sequence length: s

The number of attention heads: n

Hidden state size of one head: d

Hidden state size: h ($h = n * d$)

SwiGLU proj dim: i

Vocab size: v

Input:

X

Self Attention:

XW_Q, XW_K, XW_V

RoPE

$P = \text{Softmax}(QK^T/\sqrt{d})$

PV

AW_O

Residual Connection:

Output Shape:

(b, s, h)

FLOPs

0

$3 * 2bsh^2$

$3bsnd$

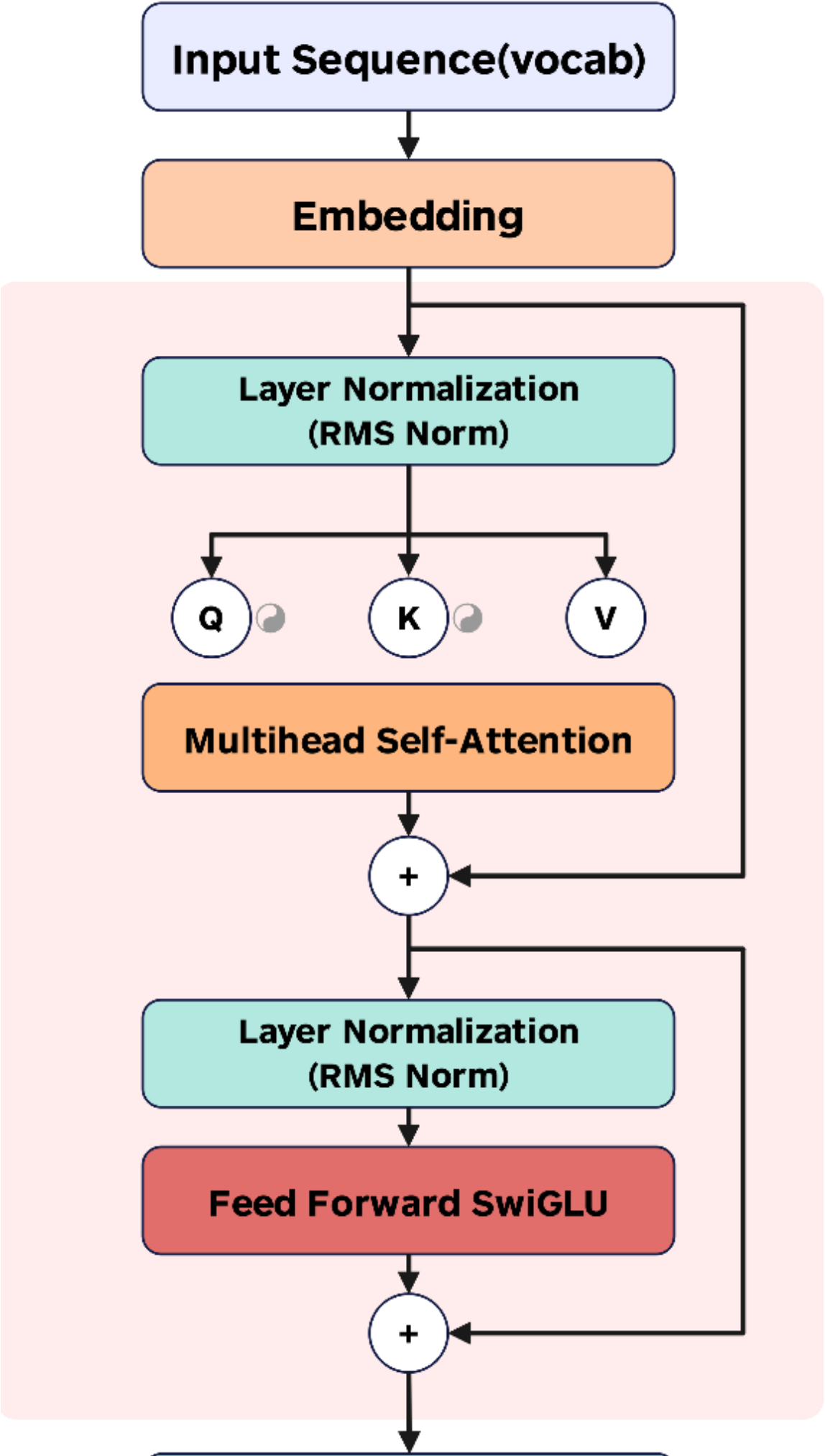
$2bs^2nd + 3bs^2n$

$2bs^2nd$

$2bsh^2$

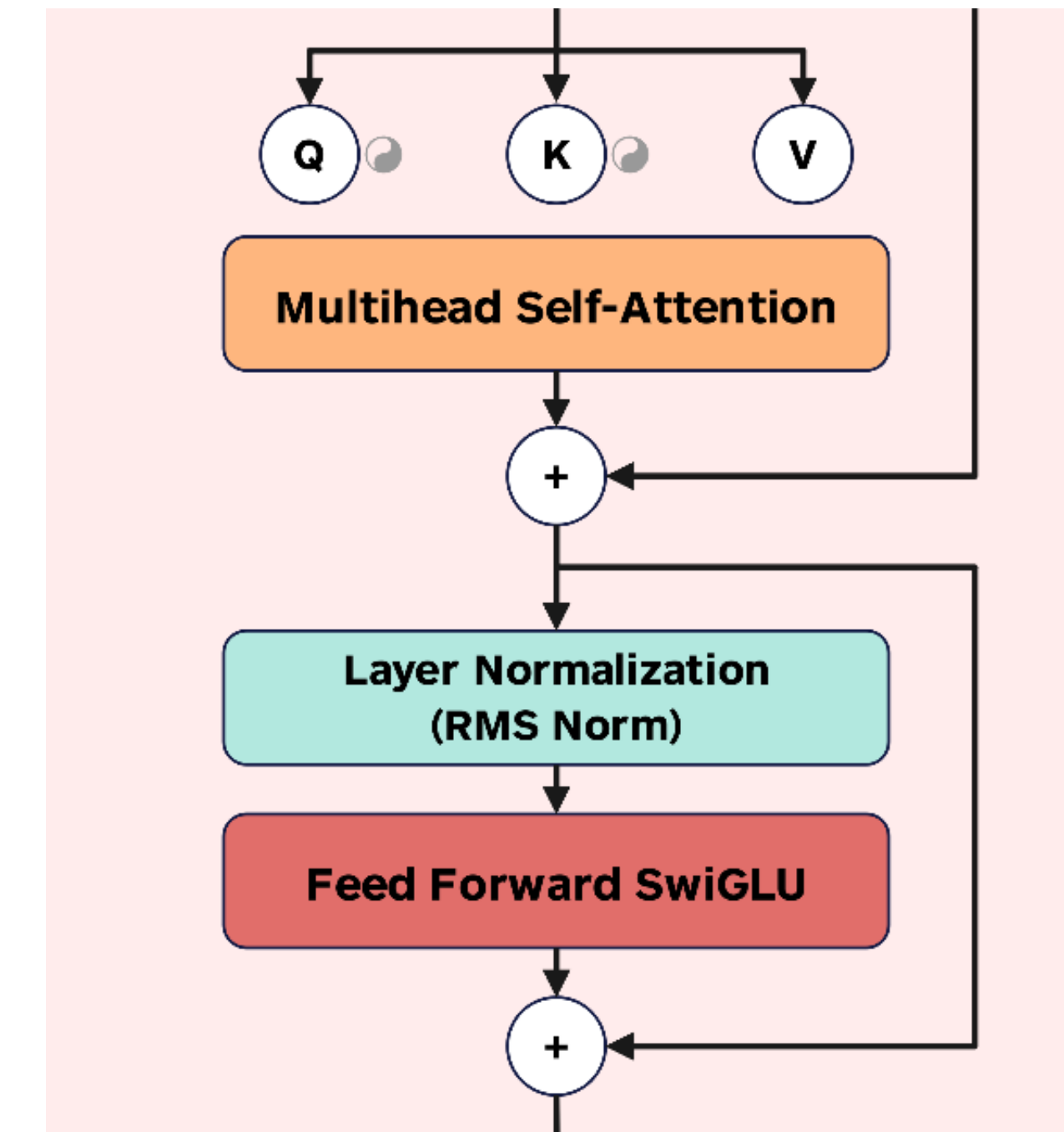
bsh

Batch size: b
Sequence length: s
of attention heads: n
Hidden state dim of one head: d
Hidden state dim: h



Output from Self Attn:	Output Shape:	FLOPs
X	(b, s, h)	
Feed-Forward		0
SwiGLU:	(b, s, i)	$2 * 2bshi$
$XW_{\text{gate}}, XW_{\text{up}}$	(b, s, i)	$4bsi$
Swish Activation	(b, s, i)	bsi
Element-wise $*$	(b, s, h)	$2bshi$
XW_{down}		$4bsh + 2bs$
RMS Norm:		
$\text{SwiGLU}(x) = \text{Swish}(xW_1 + b_1) \odot (xW_2 + b_2)$		

Batch size: b
Sequence length: s
Hidden state dim: h
SwiGLU proj dim: i



1. Calculate Root Mean Square:

- $$\text{RMS}(x) = \sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2}$$

2. Normalize:

- $$\text{RMSNorm}(x) = \frac{x}{\text{RMS}(x) + \epsilon} \cdot \gamma$$

LLama 2 7B Flops Forward (Training)

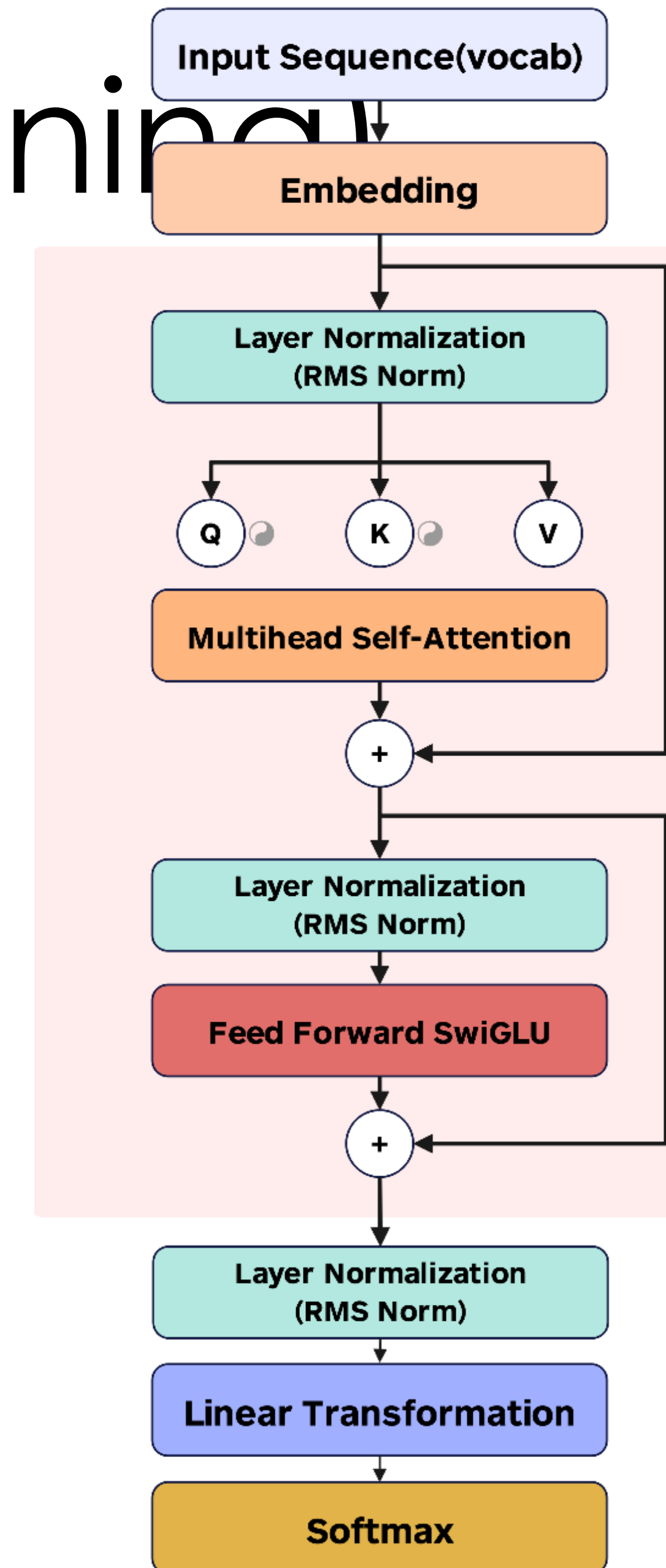
Total Flops $\approx$ #num_layers * (Attention block + SwiGLU block)

+ Prediction head

$$= \text{\#num_layers} * (6bsh^2 + 4bs^2h + 3bs^2n + 2bsh^2)$$

+ #num_layers (6bshi)

+ 2 bshv



LLama 2 7B Flops Forward Calculation (Training)

Hyperparameters:

Batch size: $b=1$

Sequence length: $s=4096$

The number of attention heads:
 $n=32$

Hidden state size of one head:
 $d=128$

Hidden state size: $h=4096$

SwiGLU proj dim: $i=11008$

Vocab size: $v=32000$

The number of layers: $N=32$

$$\text{Total Flops} \approx N * (6bsh^2 + 4bs^2h + 3bs^2n + 2bsh^2)$$

$$+ N (6bshi)$$

$$+ 2 bshv$$

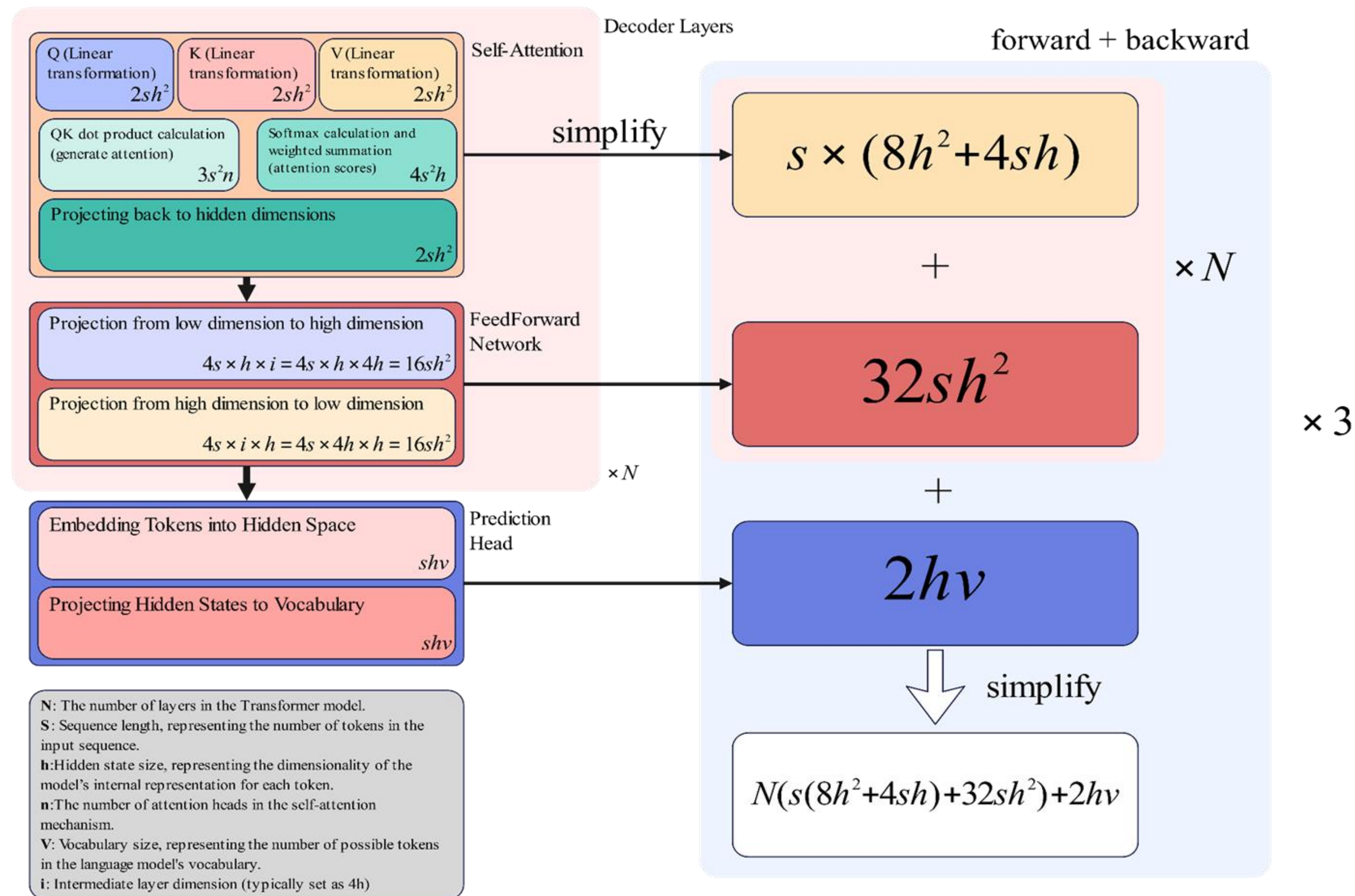
$$\approx 63 \text{ TFLOPs}$$

Flops Distribution

Training Computational Costs Breakdown:

- **Total Training TeraFLOPs: 192.17 TFLOPs**
- **FLOP Distribution by Layer:**
 - **Embedding Layer: 1.676%**
 - **Normalization: 0.007%**
 - **Residual: 0.003%**
 - **Attention: 41.276%**
 - **MLP (Multi-Layer Perceptron): 55.361%**
 - **Linear: 1.676%**

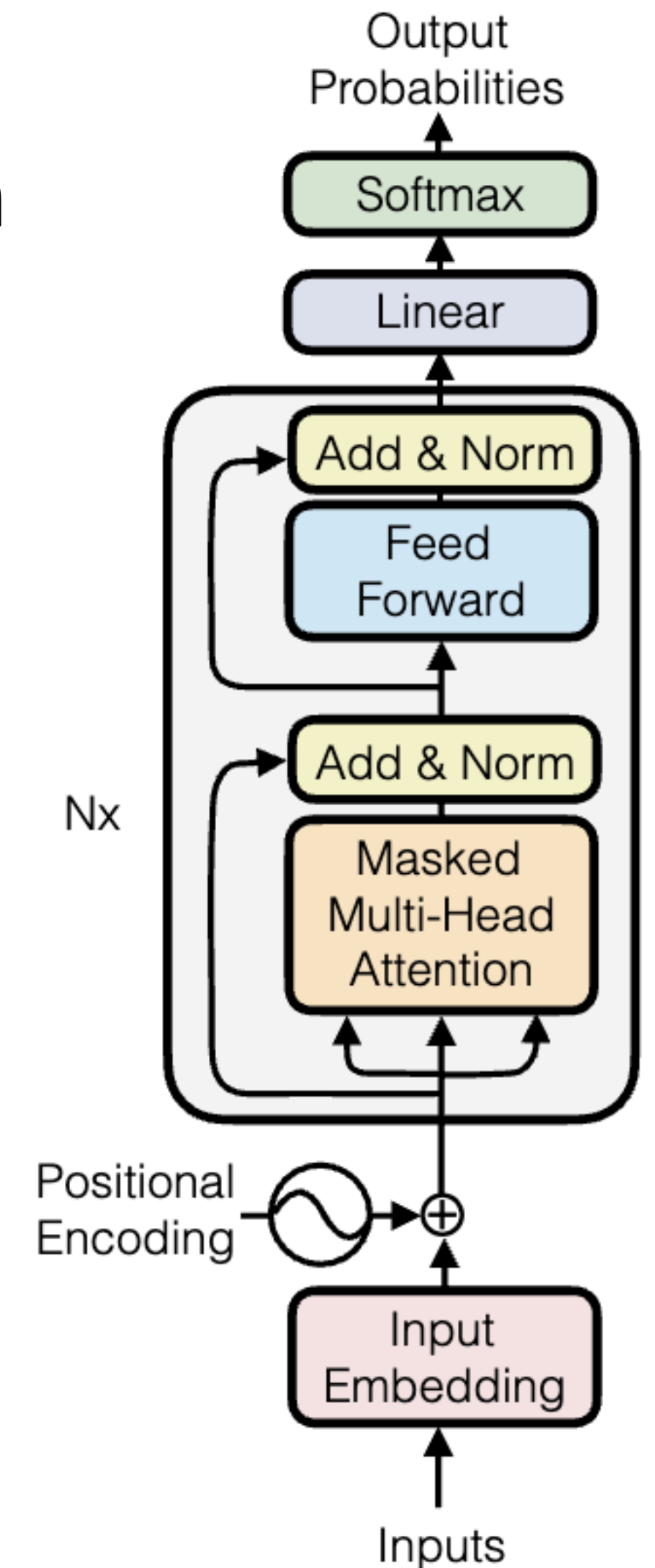
Scaling Up: Where is the Potential Bottleneck?



Connecting the Dots: Compute/Comm characteristic of LLMs

Key characteristics: compute, memory, communication

- calculate the number of parameters of an LLM?
- calculate the flops needed to train an LLM?
- calculate the memory needed to train an LLM?



Composition of Memory Usage (Training)

Model Weights

Intermediate Action Value

Optimizer States

Weight Gradients + Activation Gradients

Large Language Models

- LLM Internals
- **Scaling Law**
- Serving and inference optimization

Some Observations

- compute is a function of: h, i, b
- #parameter is a function of: h, i
- Hence: compute correlates with #parameters
 - more parameters, more compute
 - more data, more compute (of course)
- Problem: we have limited compute (\$)
- how should we allocate our limited resources:
 - Train models longer vs train bigger models?
 - Collect more data vs get more GPUs?

Motivation of Scaling Laws

- We want to know:
 - how large a model should we train...
 - How many data should we use...
 - To achieve a given performance...
 - Subject to a compute budget (\$)?

How do we do that in traditional ML: data scaling law

Input: $x_1 \dots x_n \sim N(\mu, \sigma^2)$

Task: estimate the average as $\hat{\mu} = \frac{\sum_i x_i}{n}$

What's the error? By standard arguments..

$$E[(\hat{\mu} - \mu)^2] = \frac{\sigma^2}{n}$$

This is a scaling law!!

$$\log(\text{Error}) = -\log n + 2 \log \sigma$$

More generally, any polynomial rate $1/n^\alpha$ is a scaling law

- Can we do this for transformers LLMs?
- **Unfortunately NO**

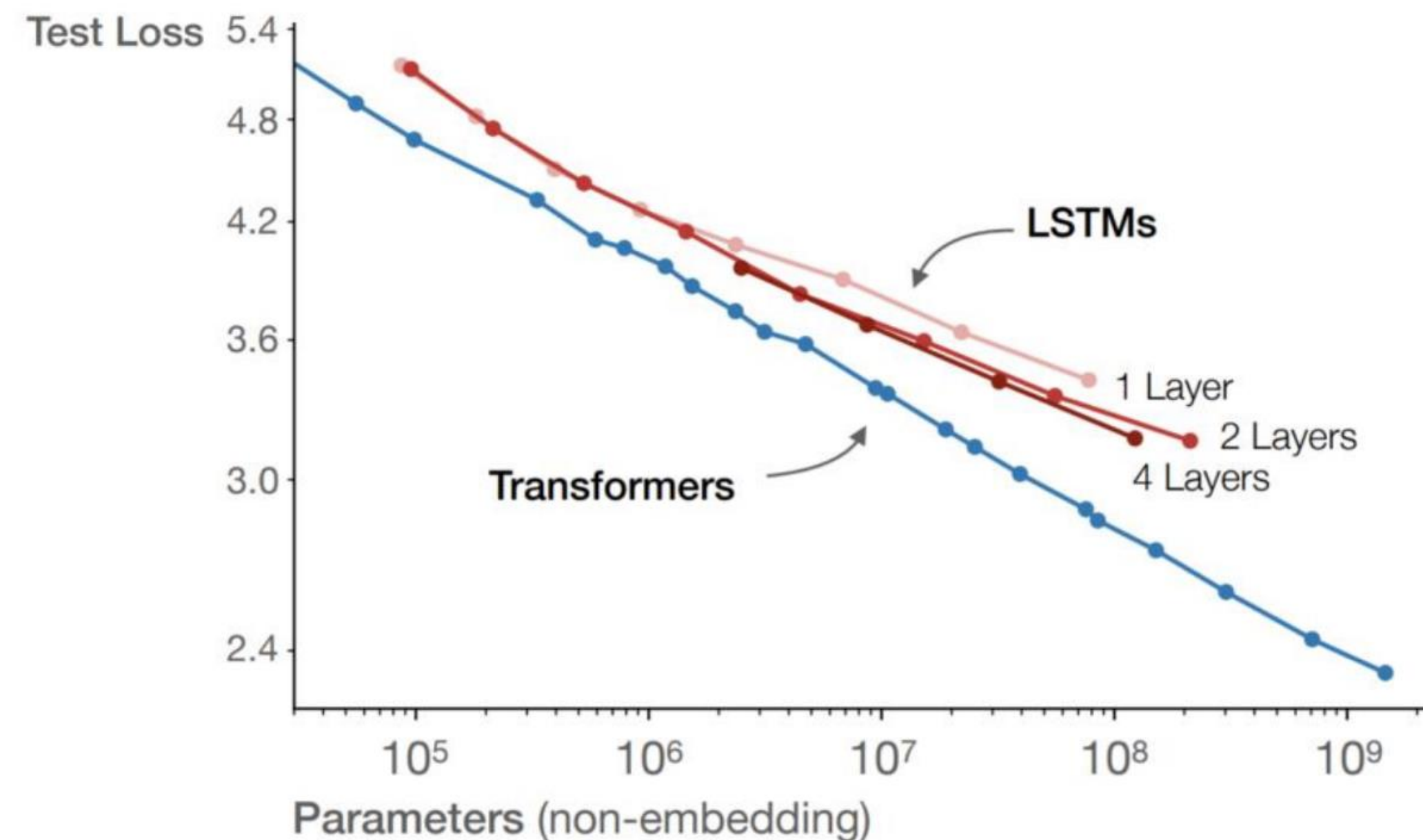
Think in this way

Mathematics vs.

Physics

Transformers vs LSTMs

- Q: Are transformers better than LSTMs?
 - Brute force way: spend tens of millions to train a LSTM GPT-3
- Scaling law way:



[Kaplan+ 2021]

The Scaling law way: Physics Way

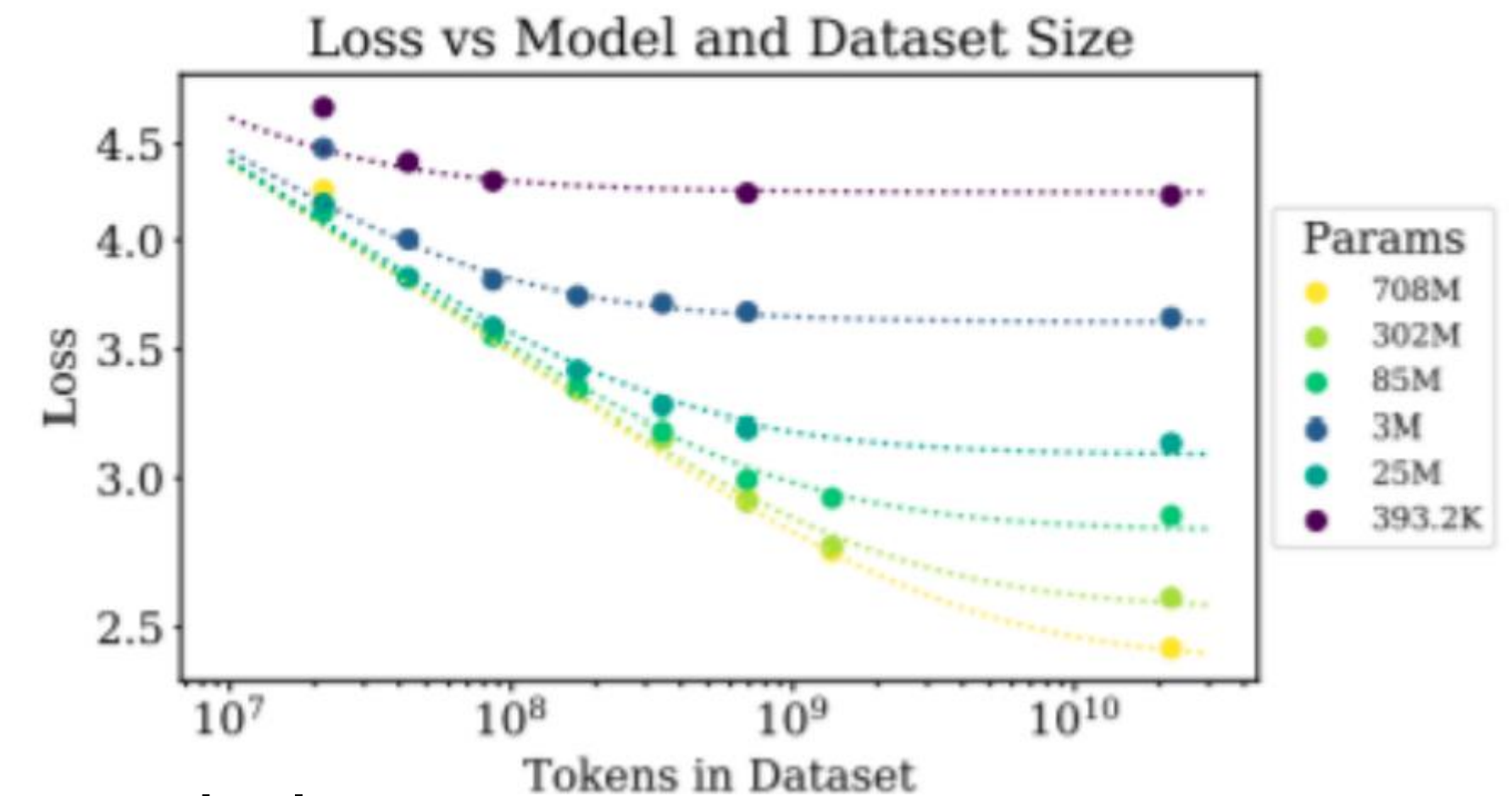
- Approach:
 - Train a few smaller models
 - Establish a scaling law (LSTM vs. transformers)
 - Select optimal hyperparam based on the scaling law prediction.
- Rationale
 - The effect of hyperparameters on big LMs can be predicted before training!
 - Optimizer choice
 - Model Depth
 - Architecture choice

Back to our problem:

- how large a model should we train...
 - How many data should we use...
 - To achieve a given performance...
 - Subject to a compute budget?
-
- Approach: estimate a law between model size data joint scaling

Model size data joint scaling

- Do we need more data or bigger models?
 - Clearly, lots of data is wasted on small models
- Joint data-model scaling laws describe how the two relate



From Rosenfeld+ 2020,

$$Error = n^{-\alpha} + m^{-\beta} + C$$

From Kaplan+ 2021

$$Error = [m^{-\alpha} + n^{-1}]^{\beta}$$

Provides surprisingly good fits to model-data joint error.

Compute Trade-offs

- Q: what about other resources? Compute vs. performance?
- For a fixed compute budget...
 - Big models that's undertrained vs small model that's well trained?
 - Solving the following optimization?

$$N_{opt}(C), D_{opt}(C) = \underset{N, D \text{ s.t. } \text{FLOPs}(N, D) = C}{\operatorname{argmin}} L(N, D).$$

Approach: empirical scaling law

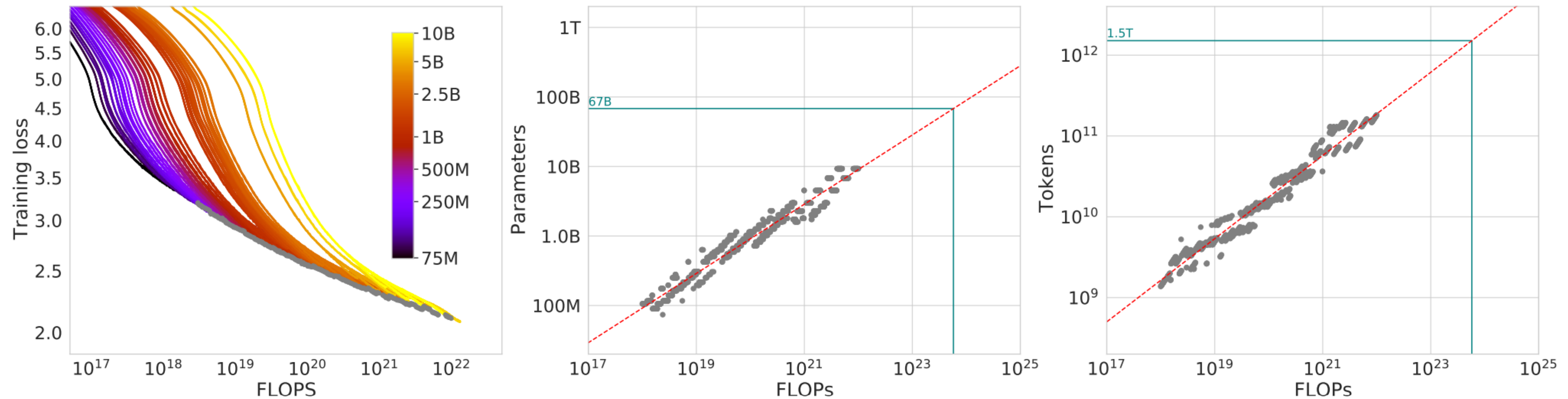


Figure 2 | **Training curve envelope.** On the **left** we show all of our different runs. We launched a range of model sizes going from 70M to 10B, each for four different cosine cycle lengths. From these curves, we extracted the envelope of minimal loss per FLOP, and we used these points to estimate the optimal model size (**center**) for a given compute budget and the optimal number of training tokens (**right**). In green, we show projections of optimal model size and training token count based on the number of FLOPs used to train *Gopher* (5.76×10^{23}).

Today's SoTA Law

$$L(N, D) = \frac{406.4}{N^{0.34}} + \frac{410.7}{D^{0.29}} + 1.69$$

Summary

- Scaling law: the physics of ML
- Scaling law marks a new era of ML research:
 - Rigorous theoretical analysis -> empirical laws
 - Exploration of different model architectures -> Scaling transformers
- Due to scaling law: ML systems become essential

PA3: Q3

You already know:

- How to estimate the number of parameters of an LLM?
 - How to estimate the flops needed to train an LLM?
 - How to estimate the memory needed to train a transformer?
-
- We will give you a scaling law and compute budget
 - Task: design your optimal LLM

Large Language Models

- LLM Internals
- Scaling Law
- **Serving and inference optimization**

Recap: Next Token Prediction

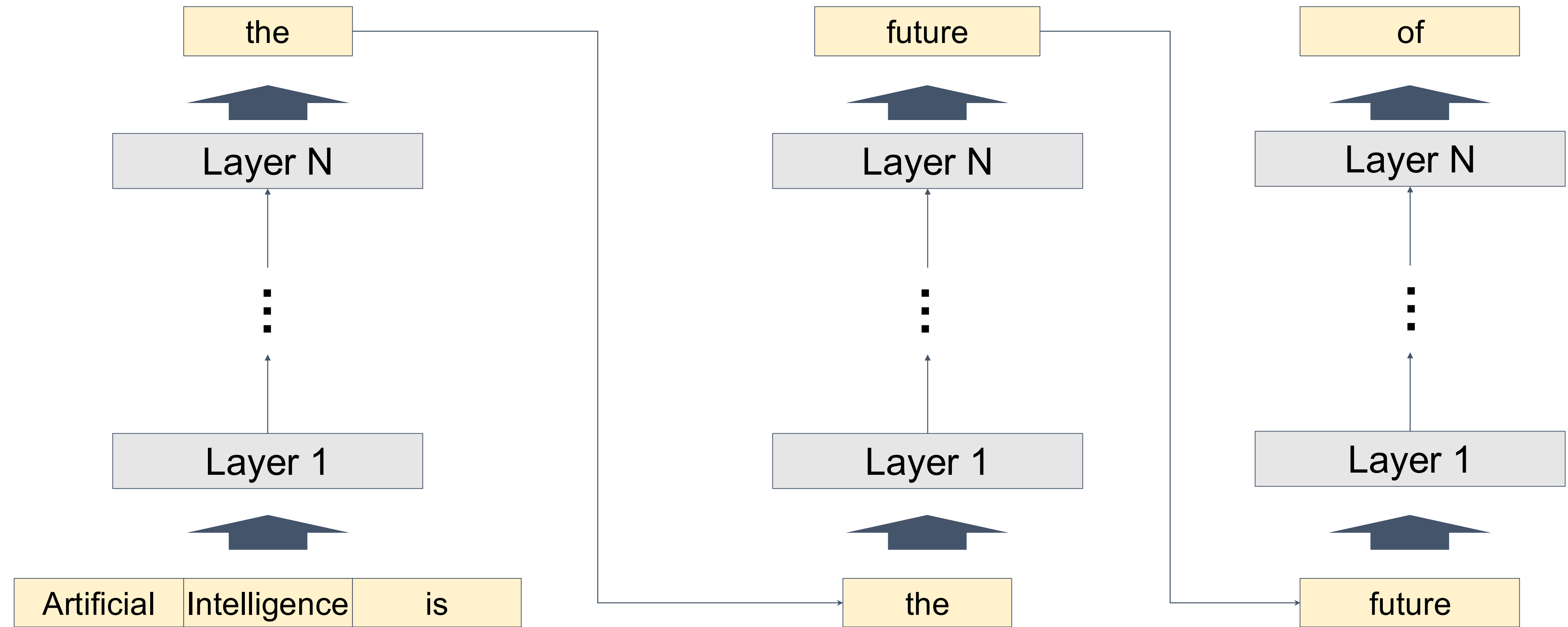
$$\begin{aligned} & \text{Probability("San Diego has very nice weather")} \\ &= P(\text{"San Diego"}) P(\text{"has"} \mid \text{"San Diego"}) P(\text{"very"} \mid \text{"San Diego has"}) \\ & \quad P(\text{"city"} \mid \dots) \dots P(\text{"weather"} \mid \dots) \end{aligned}$$

$$\text{Max Prob}(x_{1:T}) = \prod_{t=1}^T P(x_{t+1} \mid x_{1:t})$$

This is model we got — capable of
“predicting the next token”.

Inference process of LLMs

Output



Input

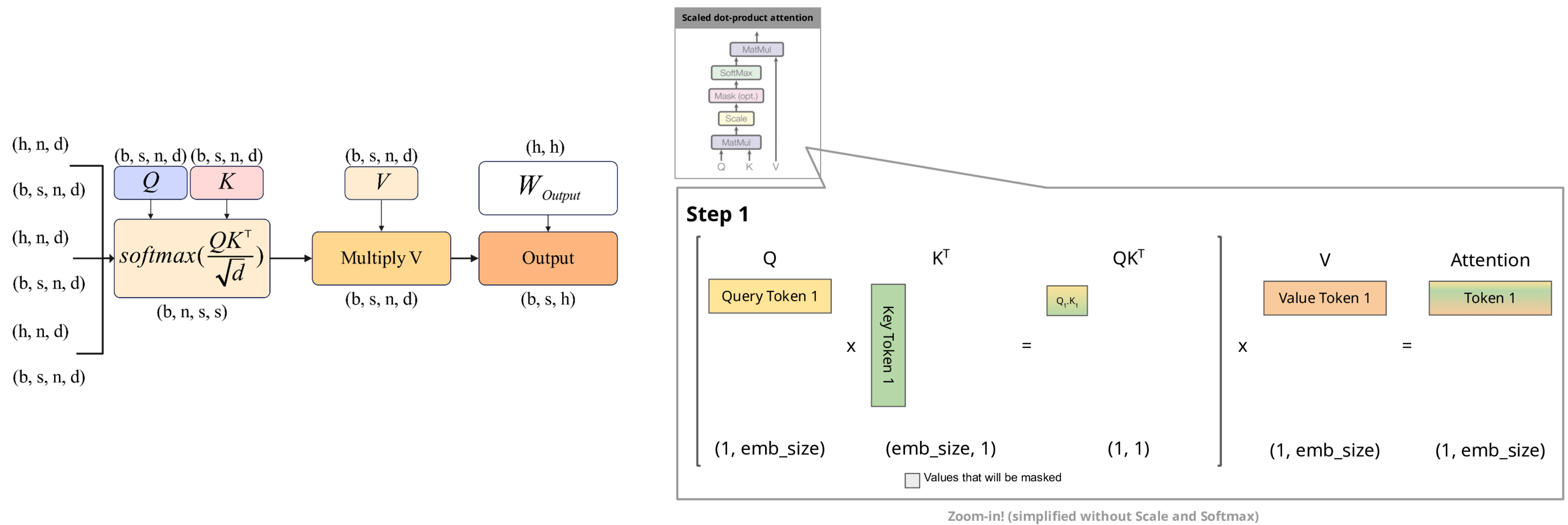
Repeat until the sequence

- Reaches its pre-defined maximum length (e.g., 2048 tokens)
- Generates certain tokens (e.g., “<|end of sequence|>”)

Generative LLM Inference: Autoregressive Decoding

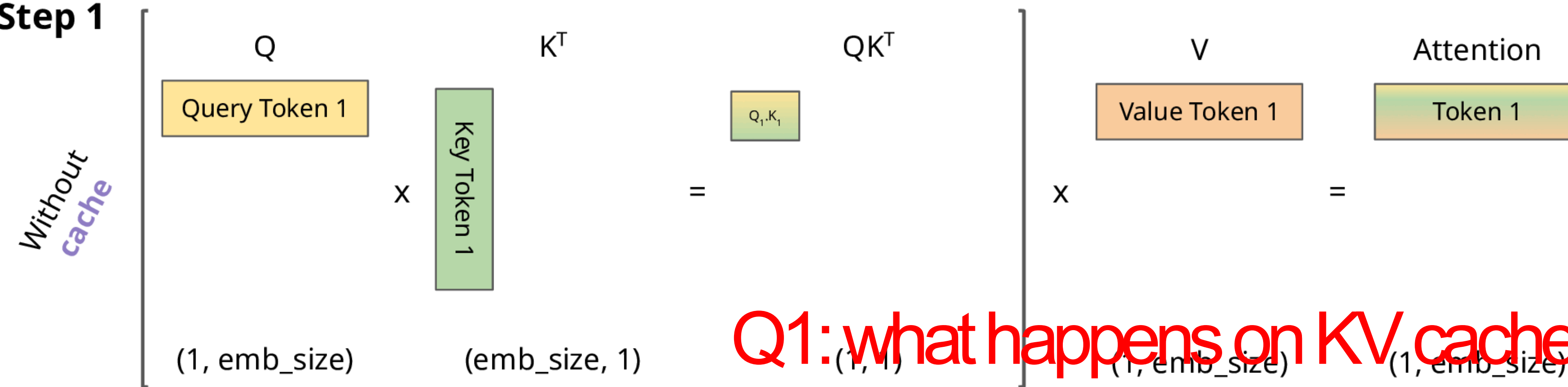
- Pre-filling phase (0-th iteration):
 - Process *all* input tokens at once
- Decoding phase (all other iterations):
 - Process a *single* token generated from previous iteration
- Key-value cache:
 - Save attention keys and values for the following iterations to avoid recomputation
 - what is KV cache essentially?

w/ KV Cache vs. w/o KV Cache



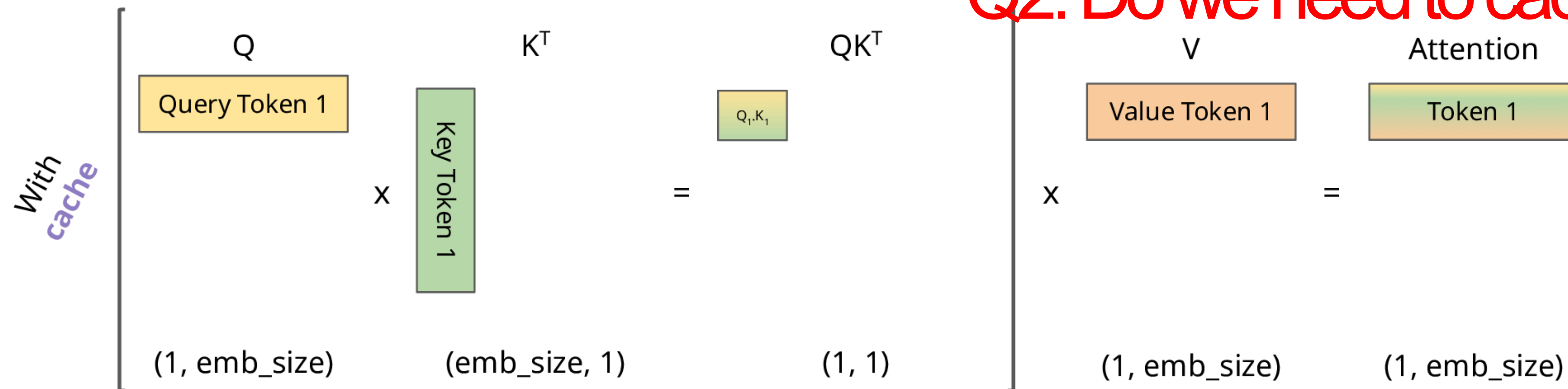
w/ KV Cache vs. w/o KV Cache

Step 1



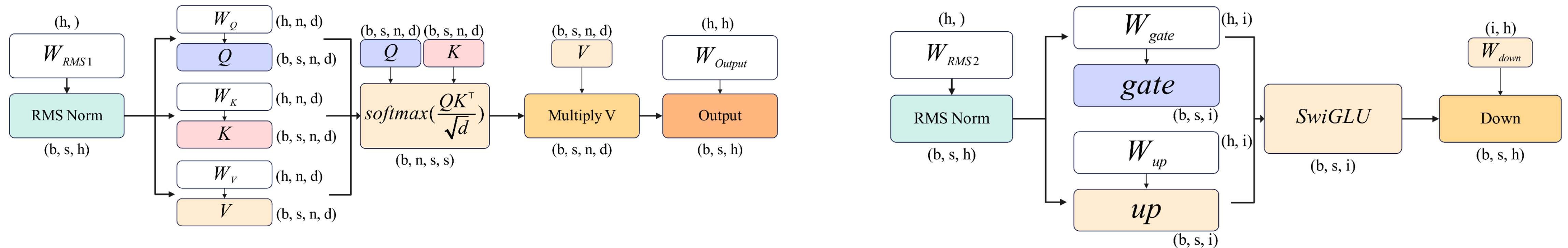
Q1: what happens on KV cache in prefill phase?

Q2: Do we need to cache Q?



Values that will be masked Values that will be taken from cache

Potential Bottleneck of LLM Inference?



- Compute:
 - Prefill: largely same with training
 - Decode: $s = 1$
- Memory
 - New: KV cache

Q? how about batch size b ?